

BNNVis: Towards Visual Analytics for Bayesian Neural Networks

Gabriel Appleby*

The National Renewable Energy Laboratory

Juliane Mueller§

The National Renewable Energy Laboratory

Malik Hassanaly†

The National Renewable Energy Laboratory

Kristi Potter¶

The National Renewable Energy Laboratory

Jen Rogers‡

Idaho National Laboratory

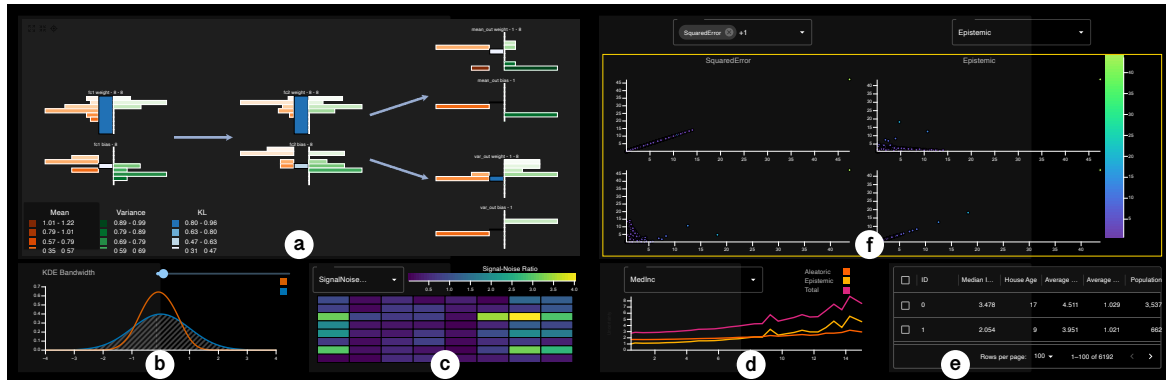


Figure 1: The BNNVis visual interface. The *Model Architecture View* (a) displays the overall architecture of the Bayesian Neural Network. The *Layer Fit View* (b) compares the empirical posterior of a selected layer to its prior. The *Neuron View* (c) displays a matrix representing the individual neurons within a layer colored by pickable metrics of interest. The *Conditional Mean View* (d) plots how different uncertainties change across the domain of input features selectable through a drop-down menu. The *Raw Data View* (e) contains a table of raw values. Finally, in the *Prediction View* (f), a Scatterplot Matrix illustrates any combination of raw data columns and BNN outputs, such as aleatoric, epistemic, and total uncertainty and error metrics relevant to the predicted data.

ABSTRACT

Bayesian Neural Networks (BNNs) offer a principled approach to modeling uncertainty in addition to providing predictions, making them particularly valuable for high-stake domains where uncertainty quantification is required. However, their adoption remains low, partly due to the difficulty in tuning and interpreting these models and their results. To address this limitation, we introduce BNNVis, a visual analytics tool designed to visualize BNNs and their results. BNNVis allows the user to understand the architecture and learned posterior weight distributions of their BNN at a glance and how these distributions differ from their prior. Additionally, the system helps them understand the distribution and magnitude of the accompanying uncertainties of the model's predictions. BNNVis provides insight into the final predictions and the model, helping practitioners tune and interpret BNNs and their results. We describe a usage scenario to demonstrate how the features of BNNVis come together to support a practitioner in using a BNN.

Index Terms: Visual Analytics, Visualization, Machine Learning, Bayesian Neural-Networks, Uncertainty Visualization

1 INTRODUCTION

Neural networks have become ubiquitous across a wide range of applications. However, standard neural networks typically pro-

duce point estimates, without expressing confidence in their predictions—a limitation that becomes essential in safety-critical applications [15] or when working with sparse or noisy datasets [16, 27]. Bayesian Neural Networks (BNNs) address this limitation by modeling uncertainty explicitly. They extend traditional neural networks by placing probability distributions over the model's weights [4], capturing *epistemic uncertainty*—uncertainty due to the dataset not adequately representing the real-world distribution of data. In addition, BNNs can incorporate *aleatoric uncertainty* [15], which represents irreducible noise inherent in the data. Utilized together, these approaches allow BNNs to provide measures of confidence with their predictions, drive data collection [22], improve features and detect out-of-distribution queries [21]. Understanding these two types of uncertainty can help a practitioner make decisions about their model. A high epistemic uncertainty might encourage a user to gather additional data or iterate on the model. A large aleatoric uncertainty might shift focus toward improving data collection processes to reduce noise through better techniques or sensors, refine the features chosen or alert the user to irreducible noise.

The Bayesian treatment of network weights also enables incorporating prior knowledge into the model parameters, offering a principled alternative to techniques such as data augmentation and regularization, and supporting continual learning scenarios [13]. Despite these advantages, BNNs remain underutilized in practice due to the challenges of interpreting their outputs and the difficulty of effectively tuning these models.

This paper introduces BNNVis, a visual analytics system designed to support practitioners in exploring and interpreting Bayesian Neural Networks. BNNVis helps users understand model behavior and prediction results through tightly integrated, interactive views. The views include an overview of the model architecture, the fit and contribution of specific layers, and the ability to contextualize the results with uncertainty and error. Through these interactions, users can analyze how uncertainty propagates through

*e-mail: Gabriel.Appleby@nrel.gov

†e-mail: Malik.Hassanally@nrel.gov

‡e-mail: Jennifer.Rogers@inl.gov

§e-mail: Juliane.Mueller@nrel.gov

¶e-mail: Kristi.Potter@nrel.gov

the network, gain a clearer understanding of how the model makes its predictions, and identify where the model or the dataset can be improved. By making BNNs more interpretable, BNNVis enables more effective use of uncertainty-aware modeling in practice.

In the following sections, we discuss the design of BNNVis, and consider a usage scenario evaluating a BNN used for home price prediction. In the demonstration, we walk through how a practitioner can leverage the views within BNNVis when tuning and interpreting a BNN and its results. The design and demonstration lead to a discussion of areas of future work that may prove useful to BNN-based visual analytics systems and understanding uncertainties associated with predictions more intuitively.

2 BACKGROUND

We offer a brief introduction to BNNs in this section, but refer the interested reader to [3] for a more thorough description.

Let $\mathcal{D} = \{\mathbf{x}_i\}$, $1 \leq i \leq N$ be a dataset consisting of N data points, each one being d -dimensional, i.e. $\forall i, \mathbf{x}_i \in \mathbb{R}^d$. A dense and deterministic deep neural network consists of L layers each containing h^l neurons. Each layer contains parameters in the form of weight matrices $\mathbf{W}^l \in \mathbb{R}^{h^l \times h^{l-1}}$ and a bias vector $\mathbf{b}^l \in \mathbb{R}^{h^l \times 1}$. Given an activation function ϕ , each layer transforms a data input $\mathbf{a}^{l-1} \in \mathbb{R}^{h^{l-1} \times 1}$ into $\mathbf{a}^l \in \mathbb{R}^{h^l \times 1}$, where $\mathbf{a}^l = \phi(\mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l)$. The parameters of the function approximated by the neural network are noted $\mathbf{w} = \{\mathbf{W}^l, \mathbf{b}^l\}_{l \in \{1, \dots, L\}}$, and layer-specific variables are noted $\mathbf{w}^l = \{\mathbf{W}^l, \mathbf{b}^l\}$. The *trainable parameters* θ of the neural net are the weights and biases of each layer, that is, $\theta = \mathbf{w}$. θ is optimized by minimizing a loss function $\mathcal{L}$ that depends on θ (through the prediction of each layer) and the dataset.

With a BNN, each element of $\mathbf{w}$ is a random variable, described by a probability density function (PDF). The objective of the BNN is to approximate the posterior $p(\mathbf{w}|\mathcal{D}) = \frac{p(\mathbf{w})p(\mathcal{D}|\mathbf{w})}{p(\mathcal{D})}$ (using Bayes theorem), where $p(\mathbf{w})$ is the *prior*, $p(\mathcal{D}|\mathbf{w})$ is the *likelihood* and $p(\mathcal{D})$ is the *evidence*. Aside from specific applications [27], the posterior is most commonly approximated via Variational Inference (VI) [13]. There, the functional form of the posterior $q_\theta(\mathbf{w})$ is chosen a priori. In this case, $\theta \neq \mathbf{w}$, but instead θ are the parameters of the posterior. In the simplest case of independent normal posterior for each variable, $\mathbf{w} \sim \mathcal{N}(\mu, \text{diag}(\sigma^2))$, where $\mu, \sigma \in \mathbb{R}^{\dim(\mathbf{w})}$, $\theta = \{\mu, \sigma\}$ and $\theta \in \mathbb{R}^{2\dim(\mathbf{w})}$. The loss function of a BNN trained with VI is most commonly formulated to minimize the Kullback–Leibler (KL)-divergence $KL(q_\theta(\mathbf{w})||p(\mathbf{w}|\mathcal{D}))$, which results in the evidence lower bound (ELBO) loss $\mathcal{L}_{ELBO} = KL(q_\theta(\mathbf{w})||p(\mathbf{w})) - \mathbb{E}_{q_\theta(\mathbf{w})}(\log(p(\mathcal{D}|\mathbf{w})))$. The first term in $\mathcal{L}_{ELBO}$ encourages the posterior to be close to the prior, while the second one penalizes the mismatch between the data and the predictions of the BNN. For BNNs that encode aleatoric uncertainties of a one-dimensional output, given a sample from $q_\theta(\mathbf{w})$ and an input $\mathbf{x}_i$, the network typically outputs a mean prediction $\mathbb{E}(y_i|\mathbf{x}_i)$ and a standard deviation $\sqrt{\text{Var}(y_i|\mathbf{x}_i)}$. The epistemic uncertainty associated with the input $\mathbf{x}_i$ is computed as $\sqrt{\text{Var}_{q_\theta(\mathbf{w})}(\mathbb{E}(y_i|\mathbf{x}_i))}$ and the aleatoric uncertainty is computed as $\mathbb{E}_{q_\theta(\mathbf{w})}(\sqrt{\text{Var}(y_i|\mathbf{x}_i)})$.

Utilizing a BNN requires a practitioner to make impactful and challenging modeling choices. First, the weight prior $p(\mathbf{w})$ plays an essential role in $\mathcal{L}_{ELBO}$ but prior knowledge is usually not available for the weights and biases themselves [2]. Second, the functional form of the posterior is often driven by computational cost considerations, which can lead to over-confident uncertainty estimates [9]. The functional form of the likelihood $p(\mathcal{D}|\mathbf{w})$ (which contains the aleatoric uncertainty modeling) also needs to be decided a priori, although it is not considered as impactful as the other two modeling choices. Typically, one adopts an iterative modeling strategy, incrementally refining these modeling choices based on the expected

behavior of the network. One of the intended uses of BNNVis is to help diagnose issues with these modeling choices by providing a visual of the BNN parameters and how they relate to the dataset $\mathcal{D}$.

3 RELATED WORK

We provide a brief overview of existing systems centered around visualizing deep learning models. A survey by Garcia et al [8], provides a taxonomy of many papers in this space, dividing them by three broad tasks (visualizing network structure, interpretation of training process, and feature understanding) and the network architecture they support (deep feedforward networks, convolutional neural networks, and recurrent neural networks). A survey by Alioglu and Sun [1] focuses more explicitly on visual analytics systems. In this section, we focus on visual analytics papers specifically, especially systems similar to ours. Visualizing a general neural network in a manner that is useful to practitioners is not an easy task. The number of neurons within modern deep-learning models makes finding effective visualizations that can give per neuron information quite difficult, as they are hard to scale visually and cognitively. Researchers tend to focus on networks such as CNNs [17, 19, 25] or RNNs [6, 7, 20, 23] where there are visual metaphors that can be relied on more easily to convey information about the network. Many of these methods [12, 14, 19, 20] utilize some sort of clustering or dimensionality reduction to organize information about neurons into more digestible and scalable chunks. Others [14, 26] rely on a more granular and explicit rendering of the computational graph agnostic of model type. There has also been a push to use tools to educate beginners in the space [17, 18, 25] training the next generation of machine learning practitioners. There is also work focused more broadly on explaining and visualizing BNNs such as Bykov et al.[5] and Grinwald et al.[10] that focus on local and global explanation methods respectively. BNNVis, while inspired by previous systems, focuses on the interpretation of Bayesian Neural Networks and their resulting uncertainties.

4 BNNVIS SYSTEM DESIGN

The system contains a number of coordinated views that come together to provide insight into the BNN and its predictions. Each view within BNNVis is sized as a percentage of the viewport and is intended for desktop displays.

4.1 Model Architecture View

The Model Architecture View is designed with the goal of helping the practitioner understand the model’s architecture, providing an overview of the learned parameters, and assisting with filtering the other model-centric views (Layer Fit View, Neuron View). As seen in Figure 1(a), the view contains a node-and-link-based graph visualization, where each “node” in the figure is an embedded view representing a layer l of our BNN, with arrows showing connections between layers. At its core, this type of diagram borrows from work such as the TensorFlow Graph Visualizer [26] but keeps to the higher-level representation of layers instead of the full computational graph to help increase understandability and interaction with the other linked views.

Each layer visualization is designed to give the practitioner information about the learned parameters within the layer, l , such as where the learned posterior has significantly deviated from the prior. A layer is represented by two instantiations of the same visualization – one for the weights $\mathbf{W}^l$ (on the top), and one for the biases $\mathbf{b}^l$ (on the bottom), labeled with their respective name and the dimensions of their matrix or array. Each instantiation has a vertically aligned bar scaled by the number of parameters in that portion of the layer ($\mathbf{W}^l$ or $\mathbf{b}^l$). The colors of the bar encode the KL divergence $KL(p(\mathbf{w})||p(\mathbf{w}|\mathcal{D}))$ between the prior $p(\mathbf{w})$ and the posterior $p(\mathbf{w}|\mathcal{D})$. The KL divergences are binned using Sturges’s

rule [24] with a maximum of nine bins. We use an ordinal version of a blue diverging colormap such that more blue represents a higher KL, and white represents a lower KL. High KL values signal layers that have learned sufficiently from the data, while very low values may suggest the prior dominates, either due to a well-chosen prior or a misaligned or overly restrictive posterior. The histogram to the left of the visualization shows the distribution of the means μ^l of all the variables $\mathbf{W}^l$ (top) or $\mathbf{b}^l$ (bottom) within the layer, and the histogram on the right shows the distribution of the variances σ^l in a similar fashion. Each histogram is colored using a diverging ordinal colormap in much the same way as the KL, with the means using orange and the variances using green colormaps to illustrate the individual buckets better.

This view illustrates the distributions of each layer's mean and variance parameters, distinguishing between the weights and biases. Selecting a layer (weight and biases separately) within the graph updates the Layer Fit View and Neuron View views with the data from the selected layer. The Model Architecture View supports a full-screen mode to allow users to explore larger networks easily by hiding all other views.

4.2 Layer Fit View

The goal of the Layer Fit View is to display a simplified view of the weight and bias posteriors compared to their priors. The $(h^l)^2$ -dimensional (for the weights) and h^l -dimensional (for the biases) prior and posteriors are represented as one-dimensional distributions, where each random variable (each weight and bias) is assumed to be drawn from the same distribution. This choice is motivated by the need for a compact representation and the fact that weights and biases posteriors within a layer are usually assumed more correlated than across layers [9]. As seen in Figure 1(b), the view contains a kernel density estimate (KDE) of the one-dimensional empirical posterior, $p(\mathbf{w}^l|\mathcal{D})$, calculated using a Epanechnikov kernel shown in orange, and a probability density function of the prior $p(\mathbf{w}^l)$, in blue with a stripped fill. A bandwidth slider is provided within the view to adjust level of smoothing within the estimation; the higher the bandwidth, the greater level of smoothing applied.

This view facilitates the comparison of the empirical posterior and prior. A wide empirical posterior indicates uncertainty in the weights, and a narrow posterior indicates confidence. Depending on how much knowledge was incorporated into the prior, the user may expect different distances between the prior and the posterior. If the prior and posterior are close but the prior was meant to be uninformative, it could indicate that the data was not as informative as the user expected. In that case, more data might be required, or that additional care should be placed in the prior definition. In turn, if the prior was meant to encode specific domain knowledge, a high distance between the prior and posterior indicates that little of the domain knowledge was retained.

4.3 Neuron View

The Neuron View (Figure 1(c)) displays a grid of rectangles for the weights or biases of the chosen layer colored using the perceptually uniform colormap Viridis. A dropdown menu allows users to choose which metric is used to color these rectangles: the signal-to-noise ratio (SNR) ($|\mu|/\sigma$), the posterior mean (μ), the posterior standard deviation (σ), or the KL divergence between the posterior and the prior. These metrics offer unaggregated insights into parameters' role in the network. For instance, the KL divergence, μ , and σ seen in Model Architecture View, can now be further investigated for outliers. Additionally, the SNR can be used as a proxy to suggest when a neuron has a low contribution either through a small μ (relatively small effect) or a large σ (relatively varied effect). In some cases, a high incidence of neurons with low SNRs could indicate that the model has not been trained with sufficient data. In net-

works with good performance, this could also indicate that weights can be pruned without affecting the network's accuracy [4]. While a useful metric for practitioners, we can offer no prescriptive recommendations about the specific SNR at which weights might be unimportant, and caution users to use their best judgment. An example of an edge case is a weight that has high SNR but low mean, which may still be indicative that the weight can be pruned [11].

4.4 Conditional Mean View

The Conditional Mean View, Figure 1(d), illustrates how different uncertainties change across the domain of input features that are set via a drop-down menu. The selected feature is again binned using Sturges's rule, and uncertainties (epistemic, aleatoric, and total) are averaged for each bin. These averages are plotted using a categorical line chart, with orange representing aleatoric, yellow representing epistemic, and cerise representing total uncertainty. This allows practitioners to understand how the uncertainties vary with respect to a specific feature [21].

4.5 Raw Data View

The Raw Data View, Figure 1(e), contains a standard table with a selection of input data and any outputs of the model, for example, in this case, epistemic, aleatoric, total uncertainty, and squared error. The table provides the ability to quickly and easily create complex and exact data filters, sort and search the table, and make selections that are then highlighted in the Prediction View. These filters make it easier for practitioners to choose exact ranges or collections and track specific instances of interest using a more familiar interface while exploring relationships within the Prediction View.

4.6 Prediction View

The Prediction View contains a Scatterplot Matrix (SPLOM) and is shown in Figure 1(f). This view plots the same data available in the Raw Data View—any of the features of the dataset, as well as aleatoric uncertainty, epistemic uncertainty, and an error metric relevant to the data. All the scatterplots can be colored by any of the columns within the dataset, allowing the user to see how input features are related to each other and how they are related to the BNN outputs. This view can help drive data collection, or model modification to mitigate the predicted uncertainties.

5 USAGE SCENARIO

To illustrate how BNNVis supports users in interpreting BNNs, we walk through a usage scenario with a data scientist who wants to understand a BNN model for predicting median house prices, based on features such as population, median income, and average number of rooms. She wants to gain a deeper understanding of the model's architecture, the learned uncertainties, and how confident the network is across different regions of the data space.

She begins her investigation in the Model Architecture View, where she examines the high-level overview of the BNN's layers. Curious about the first layer with the node colored dark blue (indicating a notably high KL divergence) she selects the embedded view representing its weight matrix. This action updates the Layer Fit View, allowing her to compare the empirical posterior with the original prior distribution, and she takes notes of their differences. She does the same to inspect other layers and notices that in some layers, the posterior closely follows the prior, prompting her to question whether those layers need further tuning or adjustments.

Once she is satisfied with her inspection across the layers, she sets the selection to be the first layer with high KL divergence, which is reflected in Layer Fit View. She now turns her investigation to the individual neurons in her selected layer. These neurons are colored by signal-to-noise, and she identifies several with some combination of a high variance and low mean, and thus colored yellow, suggesting they contribute less reliably to the model's



Figure 2: Usage scenario demonstrating the coordinated views for uncertainty and prediction analysis in the right-hand views of BNNVis. Shown clockwise, (a) the Prediction View (SPLOM) showing the relationship between epistemic uncertainty, squared error, and aleatoric uncertainty across the dataset; (b) the Raw Data View showing an interactive data table where the user can sort, filter, and select points based on features or model outputs; (c) the Conditional Mean View aggregating and binning a selected feature (here, median income) to show how aleatoric (orange), epistemic (yellow), and total (purple) uncertainty evolve across the feature space. User selections in (b) dynamically highlight corresponding points in (a), enabling precise, data-driven exploration of model behavior.

predictions. This may signal an undertrained or overparameterized network, or insufficient data coverage. Based on this, she considers whether the network architecture should be pruned, the training extended, or additional training data collected.

Her next step in exploring her BNN is to understand how uncertainty affects the predictions. She examines the Prediction View, Figure 2(a), which plots relationships between input features and model outputs such as epistemic uncertainty, aleatoric uncertainty, and squared error. Using this view, she can see how these uncertainties interact, revealing insights about model performance. For example, she observes that regions of high epistemic uncertainty tend to have a higher squared error – indicated more so in the Figure 2(a) by the color coding with epistemic uncertainty, due to the scale compression from outliers in the SPLOM.

Moving to the Raw Data View, Figure 2(b), she sorts the table to isolate a specific scenario. For example, she sorts the population in ascending order to focus on low population density regions and selects the 6 least populous entries. These points are then highlighted in the Prediction View, Figure 2(a), and summarized in the Conditional Mean View, Figure 2(c). She immediately notices that aleatoric uncertainty is higher than epistemic uncertainty, and this observation indicates to her that while the model has seen enough examples in that region to be generally confident (low epistemic uncertainty), the data itself is inherently noisy or unpredictable (resulting in higher aleatoric uncertainty). In other words, the model’s uncertainty is not due to a lack of data but instead is driven by the irreducible variability in the target variable, for example, unpredictable fluctuations in housing prices despite similar inputs.

To explore how individual features correlate with uncertainty, she returns to the Conditional Mean View, Figure 2(c). This view

aggregates and bins median income and plots the average aleatoric, epistemic, and total uncertainty. In doing so, she notices that aleatoric uncertainty is mostly flat with only a slight increase over median income values, but epistemic uncertainty increases more rapidly and sporadically. The slight increase in aleatoric uncertainty suggests there is a greater variability in the price of homes purchased by high income households. The increase in epistemic uncertainty suggests there is a lower number of data points available for high income households compared to low-income households. The Conditional Mean View provides her with an intuitive, feature-centric exploration of model behavior across the dataset.

Testing the limits of both model and tool, she next examines a second BNN trained with known training issues. While issues such as training on too few epochs would likely be detected with a standard loss curve, BNNVis affords visual diagnosis of subtler issues. For example, she explores a model trained with overly narrow priors – reflecting strong but inaccurate assumptions of the weights. In the Model Architecture View, she sees that KL divergence is abnormally high across most weights. This observation suggests that the knowledge she meant to incorporate through the prior was overconfident and erroneous. Inspecting the Layer Fit View, she notices posterior distributions with long tails – a signal that the model had to work harder to overcome a poor prior assumption. She considers whether the prior variance was too narrow, constraining the posterior unnecessarily. She flags the affected layer as a candidate for retraining with a broader or domain-informed prior. The view helps her localize the issue to specific weight groups, supporting the possibility of targeted refinement without retraining the entire network.

Collectively, these exploratory scenarios demonstrate BNNVis’s efficacy as a tool for practitioners to discover unexpected pathologies, investigate model assumptions, and refine their BNNs.

6 LIMITATIONS AND FUTURE WORK

In this section we discuss possible avenues for future work and potential limitations of BNNVis.

Further experimentation with Model Architecture View could prove particularly valuable as a stand alone component that could be utilized by other VA systems. A visualization of the fit between prior and posterior as shown in Layer Fit View within Model Architecture View would allow visualizing multiple layers’ priors and posteriors simultaneously. Additionally, extending the BNNVis to support multiple model versions at the same time would increase the tool’s effectiveness.

In this work we walked through a usage scenario that illustrated how BNNVis can support practitioners in interpreting BNNs. It is important to note that this is different from demonstrating effectiveness, a usage scenario provides no empirical evidence of effectiveness and limited proof the tool can generalize to unforeseen scenarios. Future work should include user studies that explore how BNNVis affects user decision-making, and trust in-situ, as well as whether it leads to measurable improvements in the model.

7 CONCLUSION

We presented BNNVis, a visual analytics tool designed to help practitioners interpret BNNs and their results. BNNVis allows users to see the architecture of their model, along with a summary of the learned mean and variances. They can further inspect the learned distributions of individual layers, how they differ from prior distributions, and relevant information about individual neurons. This is coupled with views that allow practitioners to understand how epistemic and aleatoric uncertainty relate to error and underlying data. We discussed several paths for future work integrating BNNs and visual analytics and walked through an example usage scenario showcasing how any analyst can use BNNVis to understand their BNN and results better.

ACKNOWLEDGMENTS

This work was authored in part by the National Renewable Energy Laboratory for the U.S. Department of Energy (DOE) under Contract No. DE-AC36-08GO28308. Funding provided by the U.S. Department of Energy Office of Science, through the Advanced Scientific Computing Research program. The views expressed in the article do not necessarily represent the views of the DOE or the U.S. Government. The U.S. Government retains and the publisher, by accepting the article for publication, acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work, or allow others to do so, for U.S. Government purposes.

REFERENCES

- [1] G. Aliçioğlu and B. Sun. A survey of visual analytics for Explainable Artificial Intelligence methods. *Computers & Graphics*, 102:502–520, 2022. doi: 10.1016/j.cag.2021.09.002 2
- [2] J. Arbel, K. Pitas, M. Vladimirova, and V. Fortuin. A primer on Bayesian neural networks: review and debates. *arXiv preprint arXiv:2309.16314*, 2023. 2
- [3] C. M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006. 2
- [4] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra. Weight uncertainty in neural network. In *International conference on machine learning*, pp. 1613–1622. PMLR, 2015. 1, 3
- [5] K. Bykov, M. M. C. Höhne, A. Creosteanu, K.-R. Müller, F. Klauschen, S. Nakajima, and M. Kloft. Explaining bayesian neural networks, 2021. 2
- [6] D. Cashman, G. Patterson, A. Mosca, N. Watts, S. Robinson, and R. Chang. RNNbow: Visualizing Learning Via Backpropagation Gradients in RNNs. *IEEE Computer Graphics and Applications*, 38(6):39–50, 2018. doi: 10.1109/MCG.2018.2878902 2
- [7] T. Dang, H. Van, H. Nguyen, V. Pham, and R. Hewett. DeepVix: Explaining Long Short-Term Memory Network With High Dimensional Time Series Data. In *Proceedings of the 11th International Conference on Advances in Information Technology*, IAIT '20. Association for Computing Machinery, New York, NY, USA, 2020. doi: 10.1145/3406601.3406643 2
- [8] R. Garcia, A. C. Telea, B. Castro da Silva, J. Tørresen, and J. L. Dohl Comba. A task-and-technique centered survey on visual analytics for deep learning model engineering. *Computers & Graphics*, 77:30–49, 2018. doi: 10.1016/j.cag.2018.09.018 2
- [9] E. Goan and C. Foakes. Bayesian neural networks: An introduction and survey. *Case Studies in Applied Bayesian Data Science: CIRM Jean-Morlet Chair, Fall 2018*, pp. 45–87, 2020. 2, 3
- [10] D. Grinwald, K. Bykov, S. Nakajima, and M. M. C. Höhne. Visualizing the diversity of representations learned by bayesian neural networks, 2023. 2
- [11] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015. 3
- [12] F. Hohman, H. Park, C. Robinson, and D. H. Polo Chau. Summit: Scaling Deep Learning Interpretability by Visualizing Activation and Attribution Summarizations. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1096–1106, 2020. doi: 10.1109/TVCG.2019.2934659 2
- [13] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun. Hands-on Bayesian neural networks—A tutorial for deep learning users. *IEEE Computational Intelligence Magazine*, 17(2):29–48, 2022. 1, 2
- [14] M. Kahng, P. Y. Andrews, A. Kalro, and D. H. Chau. ActiVis: Visual Exploration of Industry-Scale Deep Neural Network Models. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):88–97, 2018. doi: 10.1109/TVCG.2017.2744718 2
- [15] A. Kendall and Y. Gal. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in neural information processing systems*, 30, 2017. 1
- [16] M. J. Kenney, K. G. Malollari, S. V. Kalinin, and M. Ziatdinov. Predicting Battery Capacity Fade Using Probabilistic Machine Learning Models With and Without Pre-Trained Priors. *arXiv preprint arXiv:2410.06422*, 2024. 1
- [17] Y. Kim, J. Chun, S. Son, and H. Park. Toward Generalized Visualization System for Various CNN Models: Module and Layer View. *IEEE Access*, 12:158311–158321, 2024. doi: 10.1109/ACCESS.2024.3480791 2
- [18] S. Lee, B. Hoover, H. Strobelt, Z. J. Wang, S. Peng, A. Wright, K. Li, H. Park, H. Yang, and D. H. P. Chau. Diffusion Explainer: Visual Explanation for Text-to-image Stable Diffusion. In *2024 IEEE Visualization and Visual Analytics (VIS)*, pp. 96–100, 2024. doi: 10.1109/VIS55277.2024.00027 2
- [19] M. Liu, J. Shi, Z. Li, C. Li, J. Zhu, and S. Liu. Towards Better Analysis of Deep Convolutional Neural Networks. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):91–100, 2017. doi: 10.1109/TVCG.2016.2598831 2
- [20] Y. Ming, S. Cao, R. Zhang, Z. Li, Y. Chen, Y. Song, and H. Qu. Understanding Hidden Memories of Recurrent Neural Networks. In *2017 IEEE Conference on Visual Analytics Science and Technology (VAST)*, pp. 13–24, 2017. doi: 10.1109/VAST.2017.8585721 2
- [21] G. Pash, M. Hassanaly, and S. Yellapantula. a priori uncertainty quantification of reacting turbulence closure models using Bayesian neural networks. *Engineering Applications of Artificial Intelligence*, 141:109821, 2025. 1, 3
- [22] V. Rakesh and S. Jain. Efficacy of bayesian neural networks in active learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2601–2609, 2021. 1
- [23] H. Strobelt, S. Gehrmann, H. Pfister, and A. M. Rush. LSTMVis: A Tool for Visual Analysis of Hidden State Dynamics in Recurrent Neural Networks. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):667–676, 2018. doi: 10.1109/TVCG.2017.2744158 2
- [24] H. A. Sturges. The Choice of a Class Interval. *Journal of the American Statistical Association*, 21(153):65–66, 1926. doi: 10.1080/01621459.1926.10502161 3
- [25] Z. J. Wang, R. Turko, O. Shaikh, H. Park, N. Das, F. Hohman, M. Kahng, and D. H. Polo Chau. CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1396–1406, 2021. doi: 10.1109/TVCG.2020.3030418 2
- [26] K. Wongsuphasawat, D. Smilkov, J. Wexler, J. Wilson, D. Mané, D. Fritz, D. Krishnan, F. B. Viégas, and M. Wattenberg. Visualizing Dataflow Graphs of Deep Learning Models in TensorFlow. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):1–12, 2018. doi: 10.1109/TVCG.2017.2744878 2
- [27] L. Yang, X. Meng, and G. E. Karniadakis. B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. *Journal of Computational Physics*, 425:109913, 2021. 1, 2